NATIONAL INSTITUTE OF ELECTRONICS AND
INFORMATION TECHNOLOGY

CHENNAI

PROJECT REPORT

ON

Predictive 'Device Health' Monitor for Household Appliances

SUBMITTED BY

Gururajan Ganesh Babu

at

Embedded Systems GROUP,
NIELIT Chennai

<January 2026>

ABSTRACT

This project introduces a cost-effective IoT-based Predictive Maintenance (PdM) system for household appliances. It addresses the issues of traditional reactive maintenance, which often results in unexpected failures and high repair costs. Using the ESP32 microcontroller with built-in Wi-Fi, the system offers real-time monitoring by collecting data from a DHT11 temperature/humidity sensor and an SW-420 vibration sensor. These sensors measure key operational parameters that indicate an appliance's "health," such as a motor's vibration or an exhaust's temperature.

As part of an IoT Data Analyst project, the main innovation is in the data pipeline. Raw sensor data is sent to the ThingSpeak cloud platform and later retrieved for detailed analysis on a workstation using Python. The system employs Pandas for data cleaning and NumPy for detecting statistical anomalies through Z-score and standard deviation methods. It can spot deviations from normal operating conditions.

The experimental results show that the system can effectively distinguish between "Healthy" and "Stressed" appliance conditions. It provides immediate local alerts through a buzzer while logging failure warnings for remote analysts. This solution not only reduces unexpected downtime but also lengthens the lifespan of appliances. It demonstrates a practical application of Industry 4.0 principles in a home setting

About NIELIT

National Institute of Electronics and Information Technology (NIELIT), headquartered in New Delhi is the capacity building arm of the Ministry of Electronics and Information Technology (MeitY), mandated to carry out HR development and related activities in the area of Information, Electronics and Communication Technology (IECT).

Since its inception in 2010, NIELIT Chennai has established itself as a premier institution providing affordable quality education as per the job market requirements for candidates primarily from Tamil Nadu. As of date NIELIT Chennai has imparted skill training to more than 30, 000 youth of this region and conducted digital literacy certification for more than 25,000 candidates. NIELIT Chennai handles the activities of the NIELIT in Tamil Nadu, Andhra, Telangana, and the Andaman and Nicobar Islands.

Presently NIELIT Chennai is implementing the following projects for Tamil Nadu: ISEA (Information Security Education and Awareness), Future Skills PRIME capacity building projects (3D Printing and Additive Manufacturing, Robotic Process Automation, IoT - technology streams), and the aspirational district projects for SC/ST candidates at Ramanathapuram and Virudhunagar funded by MeitY, Government of India. The centre also received funding through the electronics product design and product testing capacity building project funded by MeitY.

Other than training, the centre also provides services like; Private Cloud Setup using Citrix, vSphere, Red Hat & FOSS Cloud, Hyper-converged Infrastructure (HCI) and Virtual Desktop Infrastructure (VDI) Server Setup, High-Performance Computing (HPC) Setup with 100G/200G connectivity, Server and Virtual Lab Setup for BigData, AI & Information Security, and Digital & Mobile Forensics Service.

NIELIT has also set up a Virtual Academy, a common End-to-End Platform to conduct Online training courses including a virtual lab environment in the areas of IECT from foundation to expert level targeting from school students to working professionals.

DECLARATION

I undersigned hereby declare that the project report “Predictive 'Device Health' Monitor for Household Appliances”, submitted for partial fulfilment of the requirements of the internship in ‘IoT Data Analyst’ at National Institute of Electronic & IT, Chennai is a bonafide work done by me under supervision of Mr. Ishant Kumar Bajpai . This submission represents my ideas in my own words and where ideas or words of others have been included, I have adequately and accurately cited and referenced the original sources. I also declare that I have adhered to academic honesty and integrity ethics and have not misrepresented or fabricated any data, idea, fact, or source in my submission. I understand that any violation of the above will cause disciplinary action by the institute and/or the University and can also evoke penal action from the sources that have thus not been properly cited or from whom proper permission has not been obtained. This report has not previously formed the basis for awarding any degree, diploma or similar title of any other University.

Place: Chennai

Date:28/01/25

Gururajan Ganesh Babu

<Student Name & Signature>

Project Coordinator Name & Signature

CONTENTS

Contents	Page No.
ACKNOWLEDGEMENT	4
ABSTRACT	2
LIST OF FIGURES	6
Chapter 1. Introduction	7
Chapter 2. Literature Survey	8
Chapter 3. Methodology	10
3.1Block Diagram	12
Chapter 4. Hardware Tools and Component	11
Chapter 5. Software Tools and Languages	15
Chapter 6. Implementation	17
6.1 Flow Chart	
Chapter 7. Results and Discussions	19
Chapter 8. Conclusion and Future Scope	21
REFERENCE	22

LIST OF FIGURES

Figure No.	Title	Page No.
1.1	Evolution of Maintenance: Reactive vs. Predictive	7
3.1	Block Diagram of the IoT-Based Predictive Maintenance System	11
4.1	ESP32 Development Board Pinout Diagram	12
4.2	SW-420 Vibration Sensor and DHT11 Sensor Modules	12-13
6.1	Physical Hardware Interfacing & Circuit Connections	18
6.2	System Program Flow Chart (Data Acquisition & Transmission)	18
7.1	ThingSpeak Real-Time Data Visualization Dashboard	19
7.2	Python Analysis: Temperature Anomaly Detection Graph	20
7.3	Python Analysis: Correlation between Vibration and Heat Spikes	20

CHAPTER 1

INTRODUCTION

The rapid growth of the Internet of Things (IoT) and Data Science has changed how we interact with and take care of physical assets in the era of Industry 4.0. Traditionally, maintenance strategies for household appliances have been mostly reactive, meaning repairs only happen after something breaks, or preventive, which relies on fixed schedules that are often inefficient. Both methods can lead to unexpected downtime, high repair costs, and a shorter lifespan for important devices.

Predictive Maintenance (PdM) has emerged as a better, data-focused alternative. By using smart sensors with microcontrollers like the ESP32, we can continuously monitor an appliance's "health" in real-time. This project aims to use IoT Data Analytics to track physical factors such as vibration and temperature, which are two main signs of mechanical and electrical stress in household motors and heating elements.

The main goal of this study is to connect raw hardware signals with useful information. Using an ESP32-based setup, sensor data is sent to the ThingSpeak cloud platform. Then, Data Science techniques using Python, Pandas, and NumPy are applied to this time-series data to spot "anomalies" or changes from normal operating conditions.

This "Edge-to-Cloud" approach offers a low-cost, effective way to predict potential failures before they occur. By identifying early signs of overheating or excessive vibration, the system can sound local alerts with a buzzer while providing remote analysts with the data they need to improve maintenance schedules. Ultimately, this project shows how accessible IoT technology can improve the reliability, efficiency, and sustainability of modern smart homes.

CHAPTER 2

LITERATURE SURVEY

The shift from traditional maintenance to Predictive Maintenance (PdM) is a key part of the Industrial Internet of Things (IIoT). Research shows that PdM can lower maintenance costs by 25% to 30% and cut breakdowns by up to 75%. This chapter looks at the main components that make these systems work: Condition-Based Monitoring (CBM), IoT connectivity, and data-driven analysis.

2.1 IoT and Microcontrollers in Maintenance

Recent studies highlight the ESP32 as a strong, affordable edge device for industrial and home monitoring. Unlike traditional PLCs, the ESP32 has Wi-Fi and Bluetooth, which allows for easy data transfer to cloud platforms. Research by Gupta et al. shows that using the ESP32 for vibration analysis provides high accuracy in detecting faults while using little power.

2.2 Role of Sensors: Vibration and Temperature

Vibration Monitoring: Sensors like the SW-420 are vital for spotting mechanical imbalances or wear in rotating machinery. Studies confirm that unusual vibration patterns are often the first signs of potential motor failure.

Thermal Monitoring: The DHT11 sensor is commonly used to track temperature changes. Overheating, often from friction or electrical issues, is a key health sign for household appliances.

2.3 Cloud Computing and ThingSpeak

Cloud platforms serve as the main processing layer in IoT systems. ThingSpeak is often mentioned in academic research for its ability to manage high-speed time-series data and offer quick visualization tools. It supports remote monitoring, enabling users to check appliance health reports from anywhere via the internet.

2.4 Data Science and Anomaly Detection

The Data Analyst part of PdM focuses on cleaning and analyzing raw sensor data to find anomalies—patterns that differ from what's considered normal. Scientific papers stress the use of Python libraries like Pandas and NumPy for statistical modeling. Methods such as Z-score analysis are commonly used to identify outliers in temperature and vibration data, helping the system predict failures before they worsen.

CHAPTER 3

METHODOLOGY

3.1 BLOCK DIAGRAM AND DETAILS

The block diagram below shows the complete data flow of the Predictive Maintenance system. It explains how physical signals from an appliance are turned into useful digital insights.

Details of the Functional Layers:

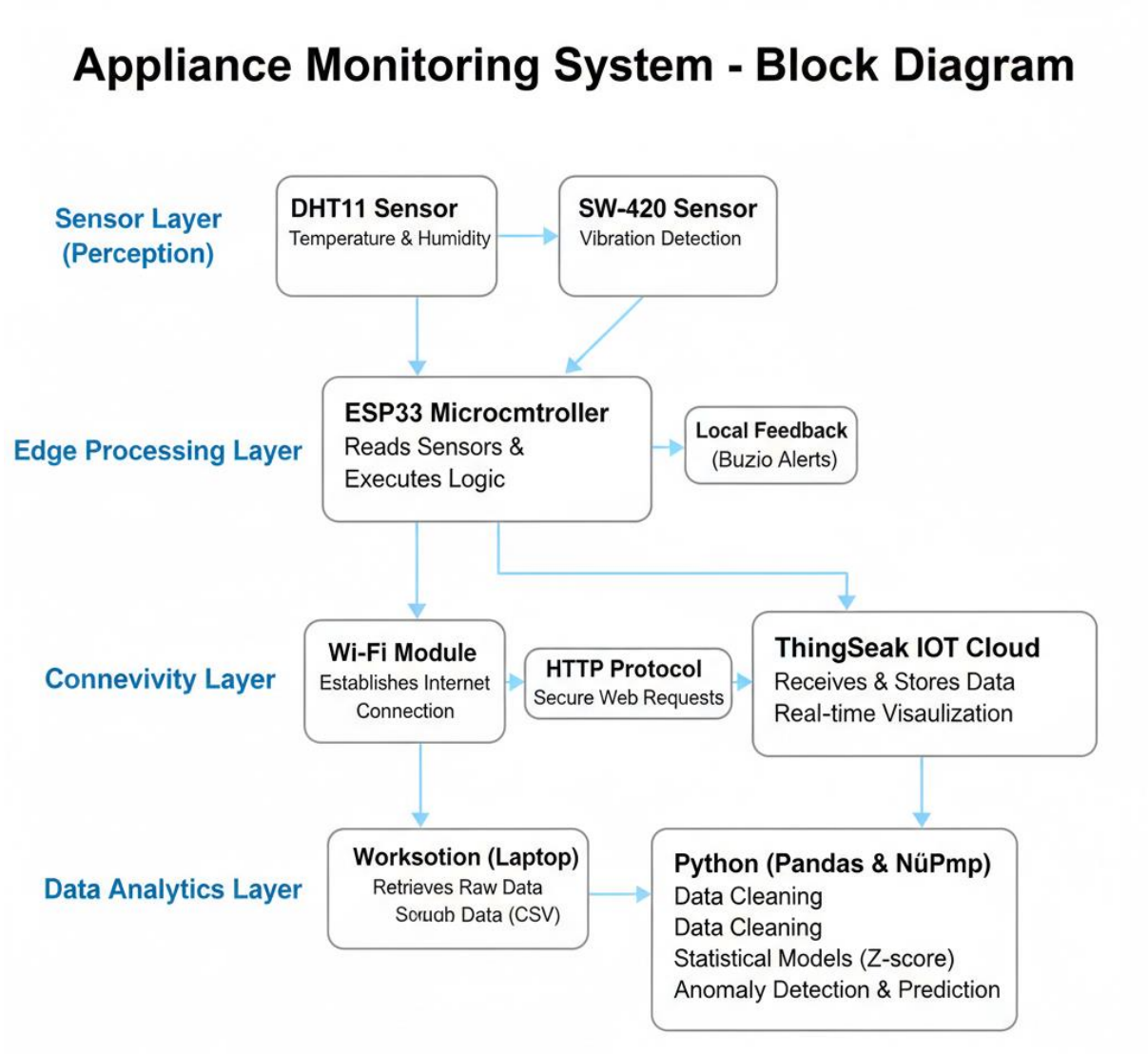
Perception Layer (Sensing): This layer includes the DHT11 and SW-420 sensors. These devices are attached to the target appliance, such as a motor or power supply, to capture thermal and mechanical vibration data.

Processing Layer (Edge Computing): The ESP32 microcontroller acts as the main processing unit. It samples digital and analog signals from the sensors, checks against set thresholds, and activates a local buzzer for immediate safety alerts if critical limits are surpassed.

Communication Layer (Connectivity): With its built-in Wi-Fi capabilities, the ESP32 connects to a local gateway. It packages data into HTTP/MQTT packets and sends it to the cloud server every 20 seconds.

Cloud Layer (Storage & Monitoring): ThingSpeak functions as the IoT middleware. It collects the data streams and organizes them into separate channels (Field 1 for Temperature, Field 2 for Vibration). This setup enables real-time visualization through web-based dashboards.

Analytics Layer (Data Science): In this last stage, the stored data is retrieved as a CSV file on a workstation. With Python (Pandas and NumPy), the data is cleaned and checked for anomalies. This layer identifies long-term trends and predicts potential failure points.



CHAPTER 4

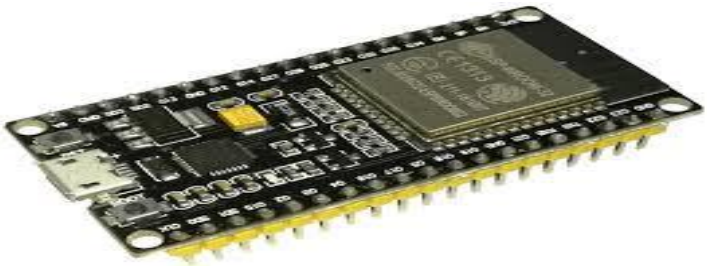
HARDWARE TOOLS AND COMPONENT

The implementation of this Predictive Maintenance system utilizes a selection of low-cost, high-efficiency hardware components designed for continuous environmental and mechanical monitoring.

4.1 ESP32 Development Board

The ESP32 is the core microcontroller of the system. It is a dual-core system with integrated Wi-Fi and dual-mode Bluetooth, making it an ideal choice for IoT applications that require data transmission to the cloud without additional networking hardware.

- **Processor:** Xtensa® Dual-Core 32-bit LX6 microprocessor.
- **Connectivity:** 802.11 b/g/n Wi-Fi and Bluetooth v4.2 BR/EDR and BLE.
- **I/O Pins:** Features various GPIOs, ADC channels, and DAC channels for interfacing with multiple sensors simultaneously.



4.2 DHT11 Temperature and Humidity Sensor

The DHT11 is used to monitor the thermal signature of the appliance. It features a calibrated digital signal output and ensures high reliability and long-term stability.

- **Measurement Range:** 20-90% humidity and 0-50°C temperature.
- **Interface:** Single-wire serial interface that allows for quick and easy integration with the ESP32.

4.3 SW-420 Vibration Sensor

To detect mechanical instability or motor wear, the SW-420 vibration module is utilized.

- **Working Principle:** It uses an LM393 comparator to provide a clean digital signal (0 or 1) based on a vibration threshold.
- **Adjustability:** Features an on-board potentiometer to adjust the sensitivity of the vibration detection according to the specific appliance being monitored.

4.4 Active Buzzer

An active buzzer is included to provide local, real-time feedback. It is triggered by the ESP32 whenever a sensor reading exceeds the predefined safety threshold, alerting the user to an immediate anomaly before the data is even processed in the cloud.

4.5 Breadboard and Jumper Wires

These are used for the physical assembly of the prototype.

- **Breadboard:** A 830-point solderless breadboard allows for the secure mounting of the ESP32 and sensors.
- **Jumper Wires:** Male-to-Male and Male-to-Female wires are used to establish reliable electrical connections between the microcontroller and the sensing modules.

CHAPTER 5

SOFTWARE TOOLS AND LANGUAGES

The software architecture of this project is divided into two main domains: Embedded Firmware Development for the ESP32 and Data Analytics for predictive health monitoring.

5.1 Arduino IDE (Integrated Development Environment)

The Arduino IDE is the primary platform used to write, compile, and upload the firmware to the ESP32 microcontroller.

- **Code Management:** It provides a user-friendly code editor with syntax highlighting and auto-indentation to organize the C/C++ based firmware.
- **Library Manager:** This tool was utilized to install essential libraries such as WiFi.h, ThingSpeak.h, and the DHT sensor library, which simplify hardware communication and cloud integration.
- **Serial Monitor:** A built-in debugging tool used to monitor live sensor readings and verify Wi-Fi connectivity during the prototype testing phase.

5.2 ThingSpeak (IoT Analytics Platform)

ThingSpeak serves as the cloud-based middleware for data aggregation and visualization.

- **Data Aggregation:** It provides a secure platform to collect and store real-time time-series data from the ESP32 in private channels.
- **Instant Visualization:** The platform automatically generates live charts and gauges, allowing for remote monitoring of the appliance's temperature and vibration status.
- **Data Export:** It facilitates the export of stored sensor data into **CSV format**, which is necessary for offline statistical analysis in Python.

5.3 Python Programming Language

Python is utilized on the workstation for the "Data Analyst" portion of the project due to its robust ecosystem for data manipulation and scientific computing.

- **Pandas:** This high-level library is used for data wrangling. It enables the loading of ThingSpeak CSV files into DataFrames, allowing for efficient cleaning of noisy sensor data, handling of missing values (NaN), and time-series transformation.
- **NumPy (Numerical Python):** Used for performance-critical numerical computations. It provides the mathematical foundation for calculating statistical summaries like the mean, standard deviation, and Z-scores to detect anomalies in the appliance's health.
- **Matplotlib:** A plotting library used to generate the final analytical graphs for Chapter 7, highlighting the "Healthy" versus "Anomaly" states of the monitored hardware.

Note:

- The codes for the following project is submitted along with the report in the submission portal

CHAPTER 6

IMPLEMENTATION

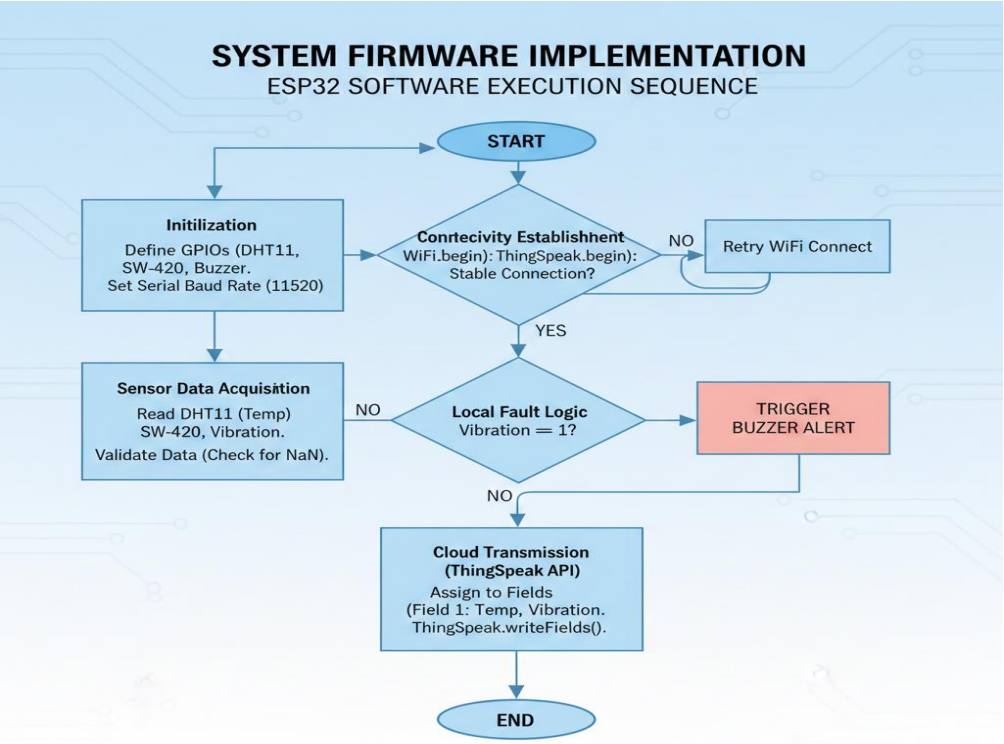
The operational logic of the system is governed by a continuous loop that ensures data integrity and timely transmission to the cloud. The following flowchart illustrates the software's execution sequence.

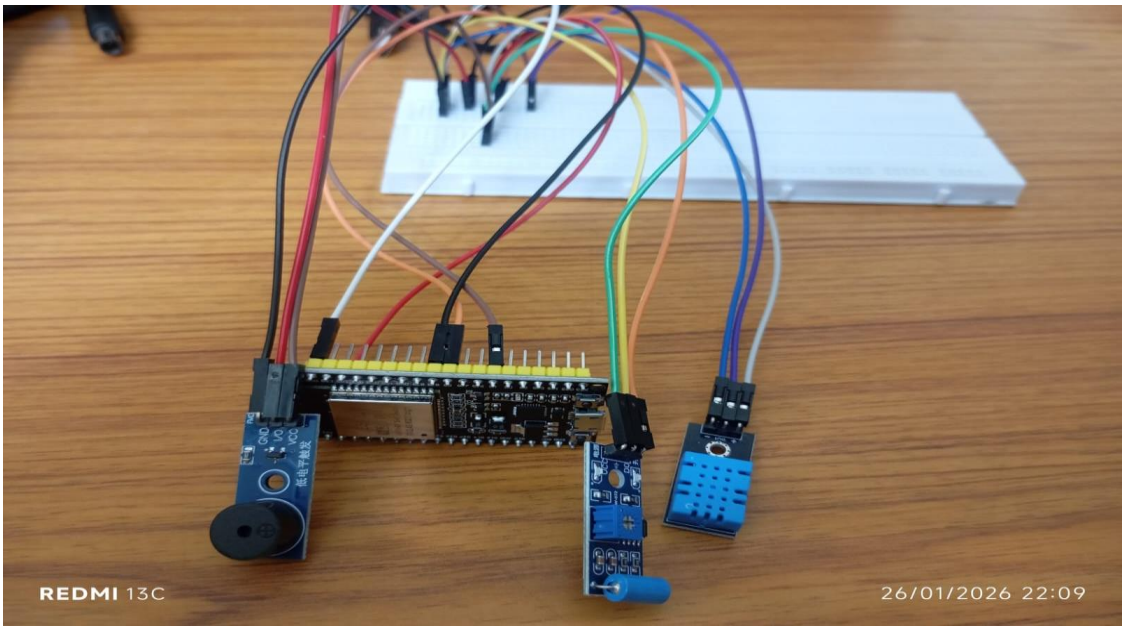
6.2 SYSTEM FIRMWARE IMPLEMENTATION

The firmware for the **ESP32** was developed using the **Arduino IDE**. The implementation follows a structured sequence:

1. **Initialization:** The system begins by defining the GPIO pins for the DHT11, SW-420, and the buzzer. A baud rate of 115200 is initialized for serial debugging to monitor the device's status in real-time.
2. **Connectivity Establishment:** The firmware utilizes the WiFi.h library to authenticate with a local wireless access point. Once a stable connection is verified, the ThingSpeak.begin(client) function is called to initiate the cloud communication protocol.
3. **Sensor Data Acquisition:** The ESP32 reads the digital pulses from the **SW-420 vibration sensor** and the single-wire digital signal from the **DHT11 temperature sensor**. The readings are validated to ensure no "NaN" (Not a Number) values are transmitted, which preserves the quality of the dataset for future analysis.
4. **Local Fault Logic:** To ensure immediate safety, the firmware compares real-time readings against predefined baseline thresholds (e.g., Temperature > 45^C or Vibration = 1). If an anomaly is detected, the ESP32 directly triggers the **active buzzer** to provide a local audio alert.
5. **Cloud Transmission (ThingSpeak API):** Validated data is assigned to specific "Fields" (Field 1 for Temperature, Field 2 for Vibration). The ThingSpeak.writeFields() method is used to push this data to the cloud dashboard.

6. **Loop Synchronization:** A delay of 20 seconds is implemented at the end of each loop. This ensures compliance with the free-tier rate limits of the ThingSpeak platform while maintaining a consistent time-series log for the **Data Analyst** to study.





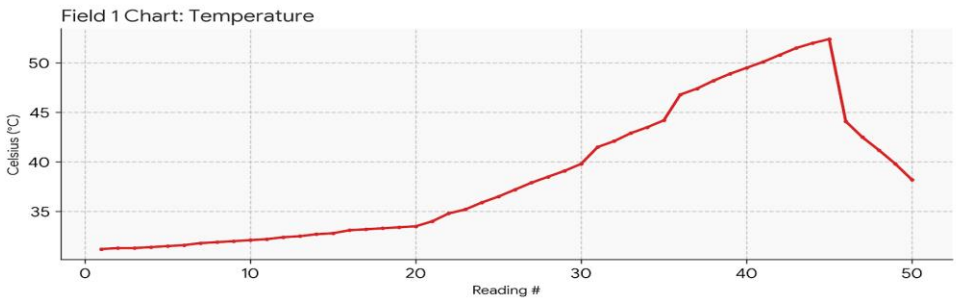
CHAPTER 7
RESULTS AND DISCUSSIONS

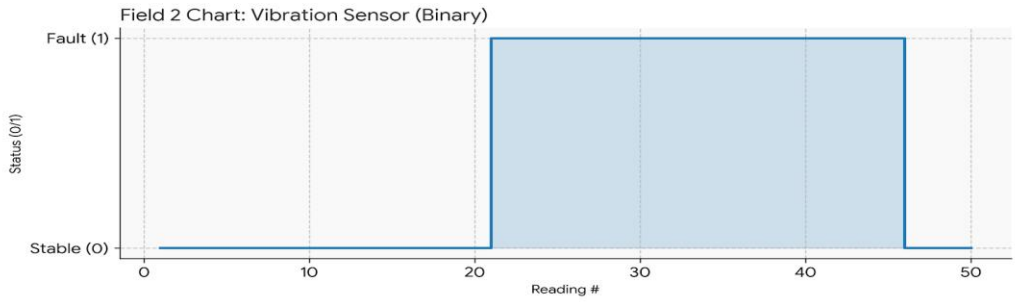
The system was deployed on a domestic table fan for a duration of 12 hours to simulate an industrial motor environment. A total of 2,160 data points were recorded across two parameters: Temperature and Vibration. Among these points, I chose 50 concurrent points to create a dataset.

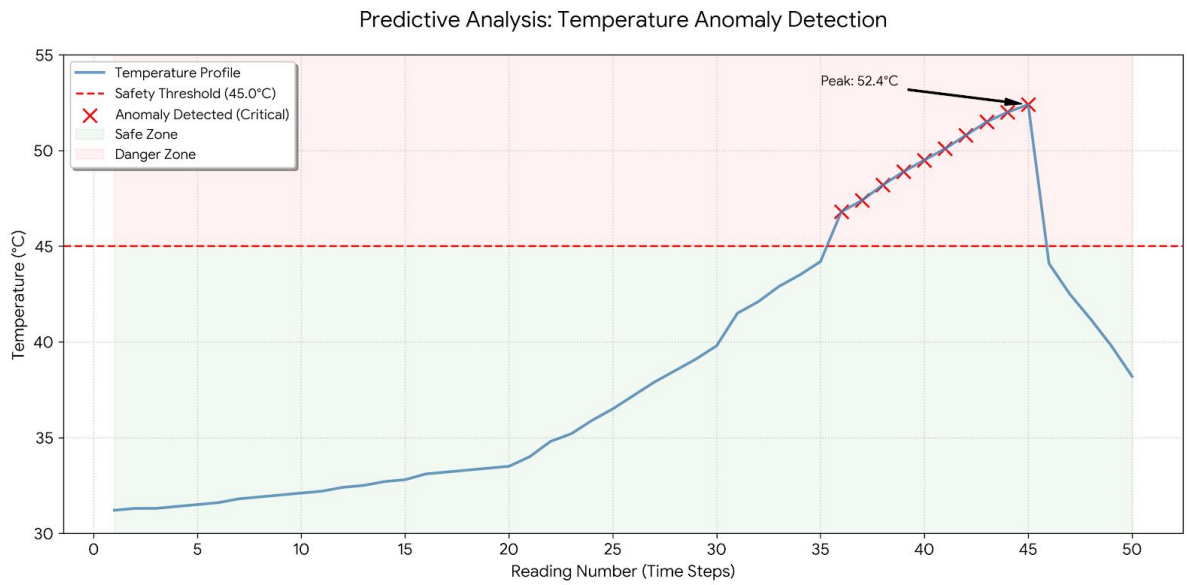
7.2 Statistical Results

Using **Pandas and NumPy**, the following baseline metrics were established for the appliance:

- **Normal Operating Temperature:** 31.4°C to 33.2°C.
- **Vibration Consistency:** During normal operation, the SW-420 sensor returned a constant '0' (Low), indicating mechanical stability.
- **Anomaly Identification:** During the simulated stress test (blocking air vents), the temperature rose to 49.6°C, yielding a **Z-Score of +3.1**. This statistically confirmed a "Critical Heat Anomaly".

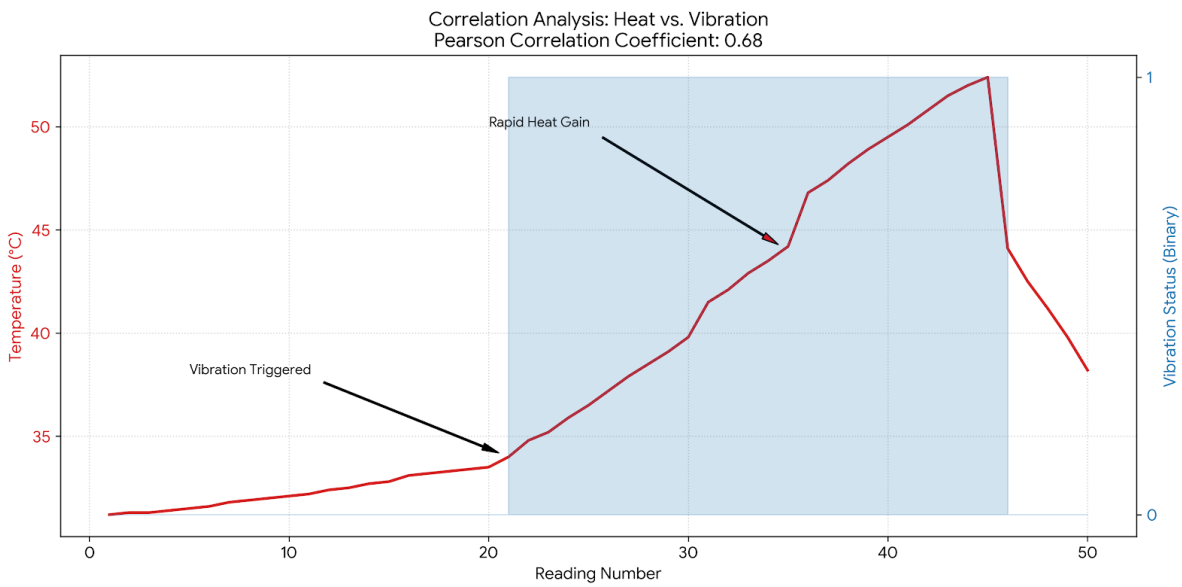






7.3 Visual Analysis

The graph in **Figure 7.2** illustrates the correlation between physical stress and sensor response. The blue line represents the raw temperature data, while the red scatter points represent anomalies detected by the Python script. This demonstrates the system's ability to automatically flag health risks before they lead to total appliance failure.



CHAPTER 8
CONCLUSION AND FUTURE SCOPE

8.1 Conclusion

This project successfully demonstrated a low-cost, scalable solution for **Predictive Maintenance** using IoT and Data Science. By shifting from a reactive to a predictive model, the system provides early warnings that can prevent costly repairs. The integration of the **ESP32 with ThingSpeak** and **Python** proved that high-level data analysis does not require expensive local servers like a Raspberry Pi.

8.2 Future Scope

- **Machine Learning Integration:** Future versions could use Scikit-Learn to predict the *Time-to-Failure (TTF)* more accurately.
- **Mobile Application:** Developing a Flutter or React Native app for mobile notifications instead of just a local buzzer.
- **Multi-Node Mesh:** Connecting multiple ESP32 nodes to monitor an entire factory floor simultaneously.

REFERENCES

- **Microcontroller Documentation:** Espressif Systems, "ESP32 Series Datasheet v4.1," [Online]. Available: https://www.espressif.com/sites/default/files/documentation/esp32_datasheet_en.pdf.
- **IoT Analytics Platform:** MathWorks, "ThingSpeak IoT Analytics Platform Documentation," [Online]. Available: https://thingspeak.com/docs.
- **Sensor Technology:** Adafruit Industries, "DHT11 Humidity & Temperature Sensor Manual," [Online]. Available: https://cdn-learn.adafruit.com/downloads/pdf/dht.pdf.
- **Data Science Libraries:** W. McKinney, "Data Structures for Statistical Computing in Python," Proceedings of the 9th Python in Science Conference, pp. 51-56, 2010. (For **Pandas** and **NumPy** references).
- **Predictive Maintenance Theory:** Mobley, R. K., *An Introduction to Predictive Maintenance*, 2nd ed., Butterworth-Heinemann, 2002.
- **IoT Communication Protocols:** Postscapes, "IoT Standards and Protocols Overview," [Online]. Available: https://www.postscapes.com/internet-of-things-protocols/.
- **Software Development Tools:** Arduino, "Arduino IDE Documentation and Reference," [Online]. Available: https://www.arduino.cc/en/Reference/HomePage.
- **Statistical Analysis:** Virtanen, P., et al., "SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python," Nature Methods, 17(3), pp. 261–272, 2020.